

2025년 게임 프로그래밍 전문가
국가기술 자격검정 필기시험문제

시험시간	2시간	형별	1-A 10매	수험번호		성명	
------	-----	----	------------	------	--	----	--

답안카드 작성 시 시험문제지 형별 누락, 수험번호 등 마킹 착오로 인한 불이익은 전적으로 수험자의 귀책사유임을 알려드립니다. 답안카드 뒷면 유의사항을 반드시 확인하시고 답안은 컴퓨터용 수성 사인펜으로만 마킹하십시오.

1 게임 프로그래밍 방법론

1. 다음 C++ 코드를 실행한 결과로 올바른 것은?

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int *p = &a;
    int **pp = &p;

    **pp = 20;
    *p = *p + 10;

    cout << a << endl;

    return 0;
}
```

- ① 10
- ② 20
- ③ 30
- ④ 40

2. 아래 코드는 표준 C++언어로 구현한 코드다. 코드의 실행결과로 올바른 것은?

```
#include <iostream>
using namespace std;

class Parent {
public:
    Parent() { cout << "Parent-"; }
    ~Parent() { cout << "~Parent-"; }
};

class Child : public Parent {
public:
    Child() { cout << "Child-"; }
    ~Child() { cout << "~Child-"; }
};

void func() {
    Child obj;
}

int main() {
    cout << "Start-";
    func();
    cout << "End";
    return 0;
}
```

- ① Start-Parent-Child-End~Child--Parent-
- ② Start-Parent-Child--Child--Parent-End
- ③ Parent-Child--Child--Parent-Start-End
- ④ Start-Child-Parent--Parent--Child-End

3. 아래 코드의 ㉠ 라인에서 calculate()함수를 올바르게 선언한 것은?

```
#include <iostream>
using namespace std;

㉠ // 함수 선언
{
    return a + b + c;
}

int main() {
    cout << calculate(1) << endl;
    cout << calculate(1, 2) << endl;
    cout << calculate(1, 2, 3) << endl;
    return 0;
}
```

출력결과
16
13
6

- ① int calculate(int a, int b = 5, int c = 10)
- ② int calculate(int a = 0, int b, int c = 10)
- ③ int calculate(int a, int b, int c = 10)
- ④ int calculate(int a = 0, int b = 5, int c)

4. 아래 코드의 ㉠ 라인에 들어갈 수 있는 클래스 생성자로 올바르게 작성된 것은?

```
#include <iostream>
using namespace std;

class Box {
private:
    double length;
    const double width; // const 멤버 변수
    double& height; // 참조 멤버 변수
public:
    ㉠ // 생성자

    double volume() {
        return length * width * height;
    }
};

int main() {
    double h1 = 1.0, h2 = 1.0, h3 = 5.0;

    Box box1(h1);
    Box box2(h2, 2);
    Box box3(h3, 3, 4);

    cout << box1.volume() << endl;
    cout << box2.volume() << endl;
    cout << box3.volume() << endl;

    return 0;
}
```

출력 결과
1
2
60

- ① Box(double& h) { length = 1; width = 1; height = h; }
- ② Box(double l, double& h) { length = l; width = 1; height = h; }
- ③ Box(double& h, double l = 2, const double w = 3) : length(l), width(w), height(h) {}
- ④ Box(double& h, double l = 1, const double w = 1) : length(l), width(w), height(h) {}

5. 다음은 C++언어로 작성된 코드로, 함수 다형성과 오버라이딩에 관한 문제다. 컴파일 할 때 에러가 발생하는 라인을 모두 찾은 것은?

```
#include <iostream>
using namespace std;

class Parent {
public:
    virtual void show() { cout << "Parent "; }
};

class Child : public Parent {
public:
    void show() { cout << "Child "; }
    void show() { cout << "Child-show"; } // ㉠ 컴파일 에러
    void display() { cout << "Display Child"; }
};

class Derived : public Parent {
public:
    void print() { cout << "Derived"; }
};

int main() {
    Parent* p = new Derived();
    p->print(); // ㉡ 컴파일 에러
    Parent* q = new Child();
    q->show(); // ㉢ 컴파일 에러
    delete p;
    delete q;
    return 0;
}
```

- ① 정상적으로 컴파일 됨
② ㉠
③ ㉠ ㉡
④ ㉠ ㉡ ㉢

6. 다음 코드에서 ㉠ 라인에 들어갈 적절한 코드와 이 코드의 실행 결과를 올바르게 짝지은 것은?(메모리 누수가 발생하면 안됨)

```
#include <iostream>
using namespace std;

int main() {
    ㉠ // 동적 메모리 할당

    for(int i = 0; i < 5; i++) {
        arr[i] = i * 10;
    }

    int* ptr = arr + 2; // arr의 세 번째 요소를 가리키는 포인터
    *ptr = 99;          // 값 변경

    ptr[-1] = 55;        // 포인터 산술 연산을 사용한 할당

    for(int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    delete[] arr;
    return 0;
}
```

- ① int* arr = new int[5];
출력 결과: 0 55 99 30 40
② int* arr = new int[5];
출력 결과: 55 99 20 30 40
③ int* arr = (int*)malloc(5 * sizeof(int));
출력 결과: 0 55 99 30 40
④ int* arr = new int[5];
출력 결과: 0 10 99 30 40

7. 다음 C++ 코드가 여러 번 실행될 때, 가장 적절한 설명은?

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main() {
    // 시드값 설정
    srand(12345);

    // 5개의 난수 생성 및 출력
    cout << "난수 5개: ";
    for(int i = 0; i < 5; i++) {
        cout << rand() % 100 << " ";
    }
    cout << endl;

    return 0;
}
```

- ① 프로그램을 실행할 때마다 다른 난수가 생성됨
② 프로그램을 실행할 때마다 항상 같은 난수 5개가 동일한 순서로 출력됨
③ 첫 번째 실행에서만 난수가 생성되고, 그 이후 실행에서는 모두 00이 출력됨
④ srand(12345)는 난수의 범위를 0부터 12345까지로 제한함

8. 다음과 같은 매크로 정의가 있을 때, 출력 결과를 올바르게 예측한 것은?

```
#include <iostream>
using namespace std;

#define SQUARE(x) x * x
#define CUBE(x) SQUARE(x) * x
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define PRINT(x) cout << #x << " = " << x << endl

int main() {
    int a = 5;
    int b = 3;

    PRINT(SQUARE(a+1));
    PRINT(CUBE(a));
    PRINT(MAX(a++, b));
    PRINT(a);

    return 0;
}
```

- ① SQUARE(a+1) = 36
CUBE(a) = 125
MAX(a++, b) = 5
a = 6
② SQUARE(a+1) = 11
CUBE(a) = 125
MAX(a++, b) = 5
a = 6
③ SQUARE(a+1) = 11
CUBE(a) = 75
MAX(a++, b) = 6
a = 7
④ SQUARE(a+1) = 11
CUBE(a) = 125
MAX(a++, b) = 6
a = 7

9. 다음 C 코드에서 ㉠에 알맞은 코드는 무엇인가?

```
#include <stdio.h>

void print_even(int num) {
    if (num % 2 == 0) printf("%d is even\n", num);
}

void print_odd(int num) {
    if (num % 2 != 0) printf("%d is odd\n", num);
}

void callback(int arr[], int size, ㉠ ) { // ㉠ 알맞은 코드는?
    for (int i = 0; i < size; i++) {
        fp(arr[i]);
    }
}

int main() {
    int arr[5] = { 1, 2, 3, 4, 5 };

    callback(arr, 5, print_even);
    callback(arr, 5, print_odd);

    return 0;
}
```

- ① void (*fp)(int) ② void fp(int)
③ int (*fp)(int) ④ void *fp(int)

10. 다음은 C++ 클래스의 오버로딩 예제이다. 출력 결과가 아래와 같이 출력되기 위해서 ㉠에 알맞은 코드는 무엇인가?

```
#include <iostream>
using namespace std;

class Calculator {
public:
    int sum(int a, int b) { return a + b; }
    double sum(double a, double b) { return a + b; }
};

int main() {
    Calculator calc;

    cout << calc.sum(3, 4) << " ";
    cout << calc.sum( ㉠ ) << endl; // 알맞은 코드를 선택
    return 0;
}
```

출력결과
7 7.5

- ① 3, 4
- ② 3.5, 4
- ③ 3.5, 4.0
- ④ '3', 4.5

11. 일반적인 서버-클라이언트 게임 아키텍처에서 서버의 주요 역할이라 볼 수 없는 것은?

- ① 게임 정보 저장 등의 데이터베이스 관리
- ② 게임 세계의 최종 상태 결정 및 유지
- ③ 게임 화면 렌더링 및 사용자 입력 처리
- ④ 게임 불법 행위 감지 및 제재 시스템 운영

12. 아래의 코드를 C++11로 컴파일을 할 때 에러가 발생하는 라인은?

```
1: #include <iostream>
2: #include <vector>
3:
4: int main()
5: {
6:     auto count = 10L;
7:     auto values = std::vector<int>{ 9, 10, 11 };
8:     auto& first = values[0];
9:     auto result;
10:    if (count > first) result = " > ";
11:    else result = " <= ";
12:    std::cout << count << result << first;
13:
14:    return 0;
15: }
```

- ① 6
- ② 7
- ③ 8
- ④ 9

13. 아래의 대화에서 (A)와 (B)에 들어갈 가장 적절한 용어의 조합은?

팀원A : 사용 중인 길찾기 검색의 CPU 부하가 너무 높은 것 같아요. CPU 부하를 (A) 할 방법이 없을까요?
팀원B : 길찾기 검색의 결과를 (B) 해두었다가, 동일한 시작점과 도착점에 대한 요청이 들어오면 검색을 다시 실행하지 않고 저장된 결과를 바로 사용하는 방법은 어떨까요?

- ① (A) 최적화(Optimization), (B) 캐싱(caching)
- ② (A) 자동화(Automation), (B) 캐싱(caching)
- ③ (A) 최적화(Optimization), (B) 인덱싱(indexing)
- ④ (A) 자동화(Automation), (B) 인덱싱(indexing)

14. 다음 중 옵저버(Observer)패턴에 대한 설명으로 가장 적절한 것은?

- ① 객체의 상태가 변경될 때 의존 객체들에게 자동으로 통지하는 일대다 의존성을 정의한다.
- ② 객체를 생성하기 위한 인터페이스를 정의하고, 어떤 클래스의 인스턴스를 생성할지는 서브클래스가 결정하도록 한다.
- ③ 기존 코드를 수정하지 않고 기능을 추가할 수 있게 해주는 구조적 패턴이다.
- ④ 알고리즘의 골격을 정의하고 일부 단계를 서브클래스에서 구현하도록 유도한다.

15. 아래의 코드를 실행하면 출력되는 값으로 맞는 것은?

```
#include <iostream>

template <typename T, typename U>
void Func(T a, U b) { std::cout << "A"; }

template <typename T>
void Func(T a, T b) { std::cout << "B"; }

template <>
void Func<int>(int a, int b) { std::cout << "C"; }

template <>
void Func<double>(double a, double b) { std::cout << "D"; }

int main()
{
    Func(1, 2);
    Func(1.5, 2.5);
    Func(1, 2.5);
    return 0;
}
```

- ① AAA
- ② BBA
- ③ CDA
- ④ CDD

16. 온라인 멀티플레이어 게임에서 '클라이언트 사이드 예측 (Client-side Prediction)은 플레이어의 입력을 서버 응답을 기다리지 않고 로컬에서 즉시 처리하여 실행 결과를 예측하는 기술입니다. 이 기술을 구현하는 주된 이유는?

- ① 서버의 처리 부담을 줄이기 위해
- ② 네트워크 지연(latency)으로 인한 사용자 경험 저하를 완화하기 위해
- ③ 클라이언트 간 공정성을 보장하기 위해
- ④ 서버 코드 복잡성을 줄이기 위해

17. 아래의 코드를 실행하면 출력되는 값으로 맞는 것은?

```
#include <iostream>
#include <list>
#include <algorithm>

int main()
{
    std::list<int> numbers = { 10, 20, 30, 40, 50 };
    auto it = std::find(numbers.begin(), numbers.end(), 30);
    numbers.insert(it, *it + 5);
    for (auto it = numbers.rbegin(); it != numbers.rend(); ++it)
        std::cout << *it << " ";
    return 0;
}
```

- ① 10 20 30 35 40 50
- ② 10 20 35 30 40 50
- ③ 50 40 35 30 20 10
- ④ 50 40 30 35 20 10

18. 아래의 코드 성능을 가장 효과적으로 개선할 수 있는 방법은?

```
1: void ProcessPhysics(std::vector<RigidBody>& bodies)
2: {
3:     for (int i = 0; i < bodies.size(); i++)
4:     {
5:         for (int j = 0; j < bodies.size(); j++)
6:         {
7:             if (i != j)
8:             {
9:                 CheckCollision(bodies[i], bodies[j]);
10:            }
11:        }
12:    }
13: }
14:
15: void CheckCollision(RigidBody& a, RigidBody& b)
16: {
17:     //계산 비용이 높은 물리 연산 수행
18: }
```

- ① 1라인의 매개변수인 bodies를 std::vector 대신 배열을 사용하여 메모리 접근 속도를 향상시킨다.
- ② 3라인과 5라인의 bodies.size()의 결과를 변수에 저장하여 루프마다 계산하지 않도록 한다.
- ③ 5라인의 j = 0 대신 모든 객체 쌍을 한 번씩만 검사하도록 내부 루프의 시작점을 j = i + 1로 변경한다.
- ④ 15라인의 매개변수 a와 b를 레퍼런스에서 포인터로 전달되도록 코드를 변경한다.

19. 아래의 C++11 코드를 실행하면 출력되는 값으로 맞는 것은?

```
#include <iostream>
#include <vector>
#include <algorithm>

int main()
{
    std::vector<int> numbers = { 5, 2, 8, 1, 9 };
    std::sort(numbers.begin(), numbers.end(),
        [](int a, int b)
        {
            return a > b;
        });
    for (int num : numbers) std::cout << num << " ";
    return 0;
}
```

- ① 5 2 8 1 9 ② 9 1 8 2 5
③ 1 2 5 8 9 ④ 9 8 5 2 1

20. C++에서 friend 선언에 관한 설명 중 옳은 것은?

- ① 클래스 A가 클래스 B를 friend로 선언하면, 두 클래스는 서로의 private 멤버와 protected 멤버에 상호 접근이 가능하다.
② 기본 클래스에서 선언한 friend 관계는 모든 파생 클래스에 자동으로 적용된다.
③ 클래스 A에서 friend로 선언된 함수나 클래스는 클래스 A의 private 멤버와 protected 멤버에 접근할 수 있다.
④ friend 선언은 클래스의 public 영역에서만 할 수 있으며, private이나 protected 영역에서는 불가능하다.

21. 표준 C언어 컴파일러로 printf()함수에 의해서 출력되는 결과값으로 올바른 것은?

```
#include <stdio.h>
int main(void)
{
    char a=128;
    printf("%d", a );
    return 0;
}
```

- ① 0 ② -1 ③ -128 ④ 128

22. 아래의 정적 라이브러리(Static Library)에 대한 설명 중 틀린 것을 고르시오.

- ① 프로그램 빌드 시에 라이브러리가 제공하는 코드를 실행 파일에 넣는 방식의 라이브러리를 의미한다.
② 컴파일의 링킹 단계에서 실행 파일에 결합 된다.
③ 실행 파일에 다 들어가기 때문에 실행 시 라이브러리가 필요 없다.
④ 같은 정적 라이브러리를 사용한 여러 프로그램을 실행할 경우 메모리를 절약한다.

23. C언어의 전역 스코프에 사용된 전처리기에 대한 예제 코드다. 다음 중 컴파일시에 오류가 발생하는 코드는 무엇인가?

- ① #ifndef MYCLASS_H_
#define MYCLASS_H_
//..중략..
#endif // MYCLASS_H_
② #define YOUR_NAME "KOCCA"
③ #define FUNC
{
printf("FUNC Name is KOCCA");
}
④ #define MAX(a,b) ((a) > (b) ? (a) : (b))

24. 아래에 설명하는 디스크 스케줄링에 맞는 기법을 고르시오.

가장 간단한 스케줄링 기법으로 요청 대기 큐에 먼저 들어온 순서대로 스케줄링 한다

- ① FCFS (First Come First Served)
② SSTF (Shortest Seek Time First)
③ SCAN (=Elevator Algorithm)
④ SLTF (Shortest-Latency-Time-First)

25. (↗)에 들어가는 적절한 것을 다음 중에서 고르시오

(↗)는 HTML 문서와 같은 리소스들을 가져올 수 있도록 해주는 프로토콜이다. (↘)는 웹에서 이루어지는 모든 데이터 교환의 기초이며, 클라이언트-서버 프로토콜이기도 합니다. 모바일 게임 서버 중 수집형 장르의 게임 서버는 (↘)을 사용하기도 한다.

- ① TCP ② UDP ③ HTTP ④ ICMP

2

게임 알고리즘과 설계

1. 다음 중 게임 내 물리 엔진을 활용하여 중력과 마찰력 등을 구현하는 이유로 가장 적합한 설명은?

- ① 캐릭터의 움직임을 자연스럽게 표현하기 위해
② 게임 속도를 인위적으로 증가시키기 위해
③ 캐릭터의 공격력을 높이기 위해
④ 게임 그래픽의 해상도를 높이기 위해

2. 다음 재귀 함수 코드에서 함수 Factorial(4)의 반환 값으로 알맞은 것은?

```
int Factorial(int n){
    if(n == 0) return 1;
    else return n * Factorial(n - 1);
}
```

- ① 4
② 6
③ 12
④ 24

3. 다음 단순 연결 리스트 노드 삭제 코드 중 옳지 않은 설명은?

```
Node* deleteNode(Node* head, int value) {
    Node* temp = head, *prev = NULL;

    if(temp != NULL && temp->value == value){
        head = temp->next;
        free(temp);
        return head;
    }

    while(temp != NULL && temp->value != value){
        prev = temp;
        temp = temp->next;
    }

    if(temp == NULL) return head;

    prev->next = temp->next;
    free(temp);
    return head;
}
```

- ① 삭제할 노드가 첫 번째 노드이면 head를 다음 노드로 변경한다.
② 삭제할 노드를 찾기 위해 리스트를 순회한다.
③ 삭제할 노드가 없는 경우 새 노드를 생성하여 연결 리스트에 추가한다.
④ 삭제할 노드의 앞 링크에 삭제할 노드의 다음 링크를 연결시키고, 해당 노드의 메모리를 해제한다.

4. 다음은 선택 정렬을 구현한 코드이다. 빈칸 (A)를 채우기 위한 코드로 알맞은 것은?

```
void selectionSort(int arr[], int size) {
    int i, j, min_idx, temp;
    for (i = 0; i < size-1; i++) {
        min_idx = i;
        for (j = i+1; j < size; j++)
            if (arr[j] < arr[min_idx])
                ( A );
        temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}
```

- ① min_idx = j
② arr[j] = min_idx
③ min_idx = arr[j]
④ j = min_idx

5. 다음은 이진 탐색(Binary Search)을 구현한 코드이다. 해당 코드가 수행하기 위한 전제 조건으로 옳은 것은?

```
int binarySearch(int arr[], int left, int right, int x) {
    if (right >= left) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == x) return mid;

        if (arr[mid] > x)
            return binarySearch(arr, left, mid - 1, x);

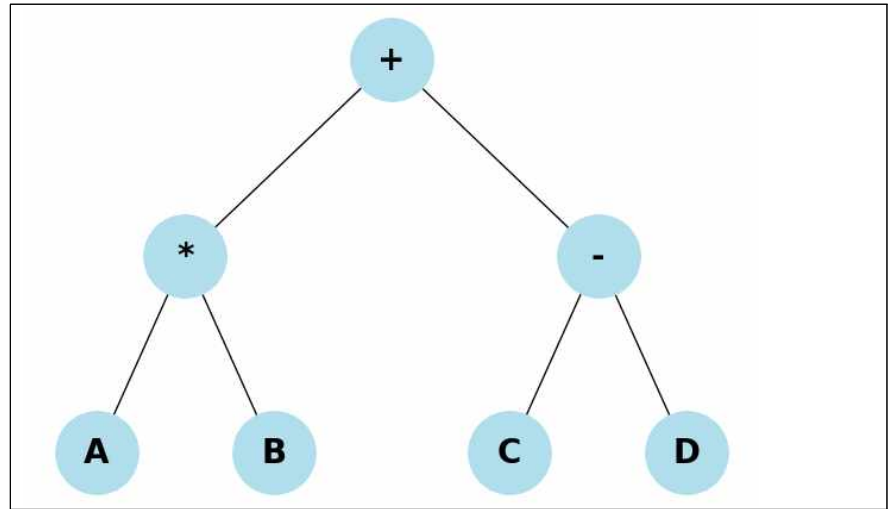
        return binarySearch(arr, mid + 1, right, x);
    }
    return -1;
}
```

- ① 배열의 모든 요소는 중복되지 않아야 한다.
- ② 배열이 반드시 정렬되어 있어야 한다.
- ③ 배열 크기가 홀수여야 한다.
- ④ 배열의 요소가 음수가 아니어야 한다.

6. 다음 중 그래프 탐색 알고리즘인 깊이 우선 탐색(DFS)과 너비 우선 탐색(BFS)에 대한 설명으로 옳지 않은 것은?

- ① BFS는 큐(Queue)를 사용하여 인접한 정점부터 순서대로 탐색한다.
- ② DFS는 스택(Stack) 또는 재귀(recursion)를 사용하여 한 방향으로 최대한 깊이 내려가 탐색한다.
- ③ BFS는 최단거리 탐색에 자주 활용되며, 모든 간선의 가중치가 동일할 때 특히 유용하다.
- ④ DFS는 항상 최단 경로를 보장하며, 최적 경로 탐색 알고리즘으로 주로 사용된다.

7. 다음의 수식 트리에 대한 중위 순회 결과로 옳은 것은?



- ① ABCD*--+
- ② AB*+CD-
- ③ A*B+C-D
- ④ AB*CD-+

8. 다음은 최소신장트리(MST, Minimum Spanning Tree)와 이를 구하는 알고리즘에 대한 설명이다. 이 중 옳지 않은 것은 무엇인가?

- ① 크루스칼(Kruskal) 알고리즘은 간선을 오름차순으로 정렬한 후, 사이클이 형성되지 않도록 작은 가중치의 간선부터 선택한다.
- ② 프림(Prim) 알고리즘은 임의의 정점에서 시작하여 인접한 정점 중 최소 가중치 간선을 선택해 신장트리를 확장한다.
- ③ 최소신장트리는 항상 유일한 형태로 존재하지는 않으며, 동일한 가중치의 간선이 여럿 있다면 여러 가지의 최소신장트리가 나올 수 있다.
- ④ 최소신장트리는 음수 가중치를 가진 간선을 포함할 수 없으므로, 음수 간선이 있는 그래프에서는 MST를 구할 수 없다.

9. 최단거리 그래프 알고리즘에 대한 설명 중 틀린 것은?

- ① 다익스트라 알고리즘은 음수 간선을 처리하지 못한다.
- ② 벨만-포드 알고리즘은 음수 간선을 처리할 수 있다.
- ③ 다익스트라 알고리즘은 항상 가장 긴 경로를 찾는다.
- ④ 최단거리 알고리즘은 두 노드 간 최소 거리의 경로를 찾는다.

10. UML 다이어그램의 설명 중 옳은 것은?

- ① 클래스 다이어그램은 시스템의 동적 행동을 나타낸다.
- ② 시퀀스 다이어그램은 객체 간 상호작용의 시간적 순서를 나타낸다.
- ③ 유스케이스 다이어그램은 데이터베이스 구조를 나타낸다.
- ④ 상태 다이어그램은 데이터 흐름을 나타낸다.

11. 아래의 배열의 대한 설명 중 틀린 것을 고르시오

- ① 배열은 같은 자료형의 데이터를 연속된 메모리 공간에 저장하는 자료구조이다.
- ② 배열의 인덱스는 일반적으로 0부터 시작하며, 크기가 n인 배열의 마지막 원소는 인덱스 n-1에 위치한다.
- ③ 배열에서 특정 원소에 접근하는 시간 복잡도는 O(n)이다.
- ④ 정적 배열은 크기가 고정되어 있어 선언 시 지정된 크기보다 많은 원소를 저장할 수 없다

12. 아래의 큐(Queue)에 관한 설명 중 맞는 것을 고르시오

- ① 큐의 기본 연산인 enqueue와 dequeue는 일반적으로 O(1)의 시간 복잡도를 가진다.
- ② 큐는 LIFO(Last In First Out) 방식으로 동작하는 자료구조이다.
- ③ 큐에서 가장 최근에 삽입된 데이터가 가장 먼저 제거된다.
- ④ 큐에서 데이터 검색은 O(1)의 시간 복잡도를 가진다.

13. 다음은 (↕) 정렬 알고리즘의 주요 특징이다. (↕)에 해당하는 정렬 알고리즘을 고르시오.

- 1. 배열의 처음부터 끝까지 반복적으로 인접한 두 원소를 비교한다.
- 2. 만약 앞의 원소가 뒤의 원소보다 크다면 두 원소의 위치를 교환한다.
- 3. 한 번의 전체 반복이 끝나면 가장 큰 원소가 맨 뒤로 "떠오르게" 된다.
- 4. 배열의 크기가 n일 때, 최대 n-1번의 전체 반복을 수행한다.
- 5. 구현이 매우 단순하지만, 대부분의 상황에서 다른 정렬 알고리즘보다 성능이 떨어진다.

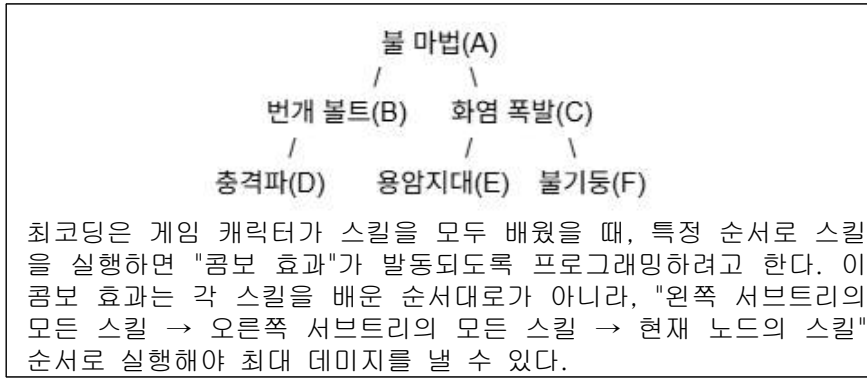
- ① 삽입 정렬(Insertion Sort)
- ② 버블 정렬(Bubble Sort)
- ③ 퀵 정렬(Quick Sort)
- ④ 선택 정렬(Selection Sort)

14. 다음은 어떤 자료 구조와 관련 있는 개념에 대한 설명이다. 이것이 무엇에 대한 설명인지 고르시오.

컴퓨터에서 데이터를 빠르게 찾기 위해 '주소 값'을 계산하는 방법이 있다. 이 방법은 데이터마다 특별한 계산식을 통해 보관 위치를 정해 주는데, 가끔 서로 다른 데이터가 같은 보관 위치를 받게 되는 문제가 생긴다.
예를 들어, '사과'와 '바나나'라는 데이터가 모두 '5번 위치'에 저장되어야 한다고 계산되면, 두 데이터를 어떻게 모두 보관할지 결정해야 한다. 이런 문제를 해결하기 위해 '연결 목록으로 여러 데이터 보관하기', '다음 빈자리 찾아가기' 같은 방법들이 사용된다. 이러한 상황이 자주 발생하면 데이터를 찾는 속도가 느려지기 때문에, 좋은 시스템은 이런 현상이 적게 일어나도록 설계한다.

- ① 데이터 손실
- ② 해시 충돌(Hash Collision)
- ③ 데이터 중복
- ④ 데드락(Deadlock)

15. 게임 개발자 최코딩은 RPG 게임의 스킬 트리 시스템을 구현하고 있다. 이 스킬 트리는 아래 그림과 같이 구성되어 있다. 아래의 스킬 트리에서 콤보 효과를 위한 최적의 스킬 발동 순서는 무엇인지 고르시오



최코딩은 게임 캐릭터가 스킬을 모두 배웠을 때, 특정 순서로 스킬을 실행하면 "콤보 효과"가 발동되도록 프로그래밍하려고 한다. 이 콤보 효과는 각 스킬을 배운 순서대로가 아니라, "왼쪽 서브트리의 모든 스킬 → 오른쪽 서브트리의 모든 스킬 → 현재 노드의 스킬" 순서로 실행해야 최대 데미지를 낼 수 있다.

- ① 전위순회 ② 후위순회 ③ 중위순회 ④ 역중위순회

게임프로그래밍 전문가 1-A형

20. 다음은 게임 개발에서 사용되는 디자인 패턴에 대한 설명이다. 이 설명이 가리키는 디자인 패턴으로 가장 적절한 것을 고르시오.

이 패턴은 게임 캐릭터의 상태(예: 대기, 걷기, 공격, 점프 등)가 변할 때 캐릭터의 행동이 변경되도록 하는 패턴이다. 플레이어가 버튼을 누를 때마다 캐릭터는 현재 상태에 따라 다르게 반응하며, 상태가 바뀌면 캐릭터의 전체 행동 방식이 바뀐다. 이것은 마치 캐릭터가 다른 모드로 전환되는 것과 같다.

- ① 옵저버 패턴(Observer Pattern) ② 싱글톤 패턴(Singleton Pattern)
③ 상태 패턴(State Pattern) ④ 팩토리 패턴(Factory Pattern)

21. 스타크래프트와 같은 전략 시뮬레이션 게임을 작성하고자 한다. 어떤 유닛(unit)에게 명령어를 연속하여 입력하여 순서대로 예약하고자 한다면 어떤 자료구조를 사용하는 것이 가장 적당한가?

- ① 스택 자료구조를 사용하여 명령어를 push하여 넣고, pop하여 각 명령어를 처리한다.
- ② 원형 큐 자료구조에 명령어를 넣고, delete하면서 명령어를 처리한다.
- ③ 트리구조에 명령어를 넣고, 루트부터 시작하여 전위 순회를 하면서 차례로 처리한다.
- ④ FSM 그래프 자료 구조를 만들어 명령어를 넣고, 순차적으로 명령어를 처리한다.

22. 다음 중 일반적인 알고리즘에 대한 설명으로 적합하지 않은 것은?

- ① 욕심쟁이 알고리즘(Greedy Algorithm) : 매 순간 가장 좋아 보이는 것을 선택하여 결정하는 알고리즘.
- ② 분할 정복 알고리즘(Divide and Conquer Algorithm) : 큰 문제를 하위 문제로 나누어 나중에 재결합하는 알고리즘
- ③ 무작위 알고리즘(Random Algorithm) : 난수를 절차적인 방법으로 이용하여 하위문제가 독립적이지 않은 문제에 대해서 해결하는 알고리즘
- ④ 근사 알고리즘(Approximation Algorithm) : 최적의 해를 구하는 대신에 충분히 좋은 것을 구하는 알고리즘

23. 다음은 자료구조 중 덱(Deque)에 대한 설명으로 틀린 것을 고르시오.

- ① 데크는 Double-Ended Queue의 줄임말인 Deque 이다.
- ② 데크는 큐의 양쪽 끝에서 삽입과 삭제가 모두 가능한 큐의 변형 자료 구조이다.
- ③ 데크는 논리적으로 배열의 시작과 끝이 연결되어 있는 것으로 간주되며 원형으로 보통 연결되어 있다.
- ④ 데크에는 입력 제한을 설정한 스크롤(SCROLL)과 출력 제한을 설정한 셸프(SHELF)가 있다.

24. 최근 국내 게임에서 블록체인 기술을 접목하여 대체불가능토큰(NFT, Non-Fungible Token)으로 게임 아이템 소유 인증을 받는 기술이 화제다. 블록체인에도 많이 쓰이는 해시 테이블(Hash Table)에 대한 설명 중 잘못된 것은?

- ① 해시 테이블은 키-값(Key, Value)으로 데이터를 저장하는 자료구조 중 하나로 내부적으로 배열(Bucket)을 사용하여 데이터를 저장하기 때문에 빠르게 데이터를 검색할 수 있는 자료구조다.
- ② 해시 함수(Hash Function)는 인자에 키(Key)를 넣어 데이터를 고정된 길이의 해시 값(Hash Value)으로 리턴해주는 함수다.
- ③ 해시 값이 충돌하는 문제를 해결하기 위해서 동일 버킷 데이터에 대해 추가 메모리를 사용하여 다음 데이터 주소를 저장하는 분리연결법(Separate Chaining)과 비어있는 해시테이블 공간을 활용하는 개방주소법(Open Addressing)을 사용한다.
- ④ 해시 테이블의 시간복잡도는 각각의 키 값에 대하여 해시 함수에 의해 고유한 인덱스를 가지고 바로 접근할 수 있어 평균 $O(1)$ 이 걸리고, 해시 충돌 경우에는 분리연결법에서 연결된 리스트들까지 검색을 해야 하므로 $O(n^2)$ 까지 증가할 수 있다.

25. 다음은 최단경로 알고리즘에 대한 설명의 일부분이다. 이 특징에 가장 가까운 알고리즘을 고르시오.

- Ⓐ 하나의 정점에서 다른 하나의 정점까지의 최단경로를 구하는 것이 아닌 다른 모든 정점까지의 최단 경로를 구하는 알고리즘이다.
- Ⓑ 우선순위 큐를 사용해서 방문하지 않은 정점들을 시작점으로부터의 거리에 따라 저장한다.
- Ⓒ 항상 하나의 시작점으로부터 그래프의 다른 모든 정점들 중에서 다음으로 가까운 정점을 선택한다.
- Ⓓ 그래프의 어떤 간선도 음수 값을 가지지 않는다.

- ① 벨만-포드(Bellman-Ford) 알고리즘
- ② 다익스트라(Dijkstra) 알고리즘
- ③ A*(A Star) 알고리즘
- ④ BFS(Breadth First Search) 알고리즘

3

게임 콘텐츠 프로그래밍

1. 다음 중 게임 개발에서 메모리 관리가 가장 중요한 이유와, 이를 효과적으로 처리할 수 있는 프로그래밍 언어에 대한 설명으로 옳은 것은?

- ① 게임의 로딩 시간을 줄이기 위해서이며, JavaScript가 가장 효과적이다
- ② 프레임 레이트를 안정적으로 유지하기 위해서이며, C++가 가장 효과적이다
- ③ 네트워크 지연을 최소화하기 위해서이며, Java가 가장 효과적이다
- ④ 그래픽 품질을 향상시키기 위해서이며, Python이 가장 효과적이다

2. 다음 코드는 어떤 알고리즘을 구현한 것인지 고르시오.

```
bool checkCollision(GameObject* obj1, GameObject* obj2) {
    float dx = obj1->x - obj2->x;
    float dy = obj1->y - obj2->y;
    float distance = sqrt(dx*dx + dy*dy);
    return distance < (obj1->radius + obj2->radius);
}
```

- ① A* 경로 탐색
- ② 구형 충돌 감지
- ③ 깊이 우선 탐색(DFS)
- ④ 쿼드 트리(Quad Tree) 분할

3. 최신 게임에서 사용되는 '레이 트레이싱(Ray Tracing)' 기술에 대한 설명으로 옳바른 것은?

- ① 3D 모델의 폴리곤 수를 실시간으로 줄여 성능을 최적화하는 기술
- ② 빛의 물리적 경로를 시뮬레이션하여 사실적인 반사, 그림자, 굴절 효과를 구현하는 기술
- ③ 2D 이미지를 3D 모델로 자동 변환하는 인공지능 기반 기술
- ④ 여러 대의 GPU를 병렬로 연결하여 렌더링 속도를 향상시키는 기술

4. 다음 중 '물리 기반 렌더링(PBR, Physically Based Rendering)'의 주요 특징이 아닌 것은?

- ① 실제 물리 법칙을 기반으로 빛과 재료의 상호작용을 시뮬레이션한다
- ② 금속성(Metallic)과 거칠기(Roughness) 값을 사용하여 재질을 정의한다
- ③ 다양한 조명 환경에서 일관된 재질 표현이 가능하다
- ④ 렌더링 속도를 높이기 위해 빛의 반사와 굴절 효과를 단순화한다

5. 게임 AI에서 사용되는 '행동 트리(Behavior Tree)'에 대한 설명으로 옳바른 것은?

- ① 플레이어의 행동을 학습하여 비슷한 패턴으로 대응하는 머신러닝 알고리즘이다
- ② 게임 캐릭터의 애니메이션 전환을 관리하는 그래프 구조이다
- ③ AI의 의사 결정 과정을 계층적으로 구성하여 복잡한 행동을 모델링하는 구조이다
- ④ 게임 내 모든 객체 간의 상호작용을 정의하는 데이터베이스 구조이다

6. 모바일 게임에서 배터리 소모를 최소화하기 위한 최적화 기법으로 가장 효과적이지 않은 것은?

- ① GPU 사용률을 줄이기 위해 화면 해상도를 동적으로 조절한다
- ② 화면 밖 객체의 업데이트를 중지하는 컬링(Culling) 기법을 적용한다
- ③ 가능한 모든 계산을 매 프레임마다 정확하게 수행한다
- ④ 애니메이션과 물리 시뮬레이션의 업데이트 빈도를 조절한다

7. 다음 중 '역운동학(Inverse Kinematics, IK)' 기술의 주요 용도로 가장 적절하지 않은 것은?

- ① 캐릭터의 발이 지형에 자연스럽게 접촉하도록 조정
- ② 캐릭터가 물체를 잡을 때 손의 위치와 방향을 조정
- ③ 캐릭터의 시선이 특정 대상을 자연스럽게 따라가도록 조정
- ④ 미리 제작된 애니메이션 시퀀스의 프레임 수를 줄여 메모리 사용량 최적화

8. 다음 HLSL 코드가 수행하는 기능으로 가장 적절한 것은?

```
cbuffer TransformBuffer : register(b0) {
    matrix World;
    matrix View;
    matrix Projection;
}

struct VertexInput {
    float4 position : POSITION;
    float4 color : COLOR;
};

struct PixelInput {
    float4 position : SV_POSITION;
    float4 color : COLOR;
    float fog : FOG;
};

PixelInput VS(VertexInput input) {
    PixelInput output;

    float4 worldPosition = mul(input.position, World);
    float4 viewPosition = mul(worldPosition, View);
    output.position = mul(viewPosition, Projection);

    float distanceToCamera = length(viewPosition.xyz);
    output.fog = saturate((distanceToCamera - 10.0f) / 40.0f);
    output.color = input.color;

    return output;
}

float4 PS(PixelInput input) : SV_TARGET {
    float4 fogColor = float4(0.5f, 0.5f, 0.5f, 1.0f);
    return lerp(input.color, fogColor, input.fog);
}
```

- ① 텍스처 매핑을 구현한 셰이더
- ② 거리 기반 안개(fog) 효과를 구현한 셰이더
- ③ 노멀 매핑을 구현한 셰이더
- ④ 그림자 매핑을 구현한 셰이더

9. 다음 HLSL 셰이더 코드가 수행하는 기능으로 가장 적절한 것은?

```
sampler2D normalMap;
struct PS_INPUT
{
    float2 Texcoord : TEXCOORD0;
    float3 ViewDirection : TEXCOORD1;
    float3 LightDirection: TEXCOORD2;
};

float4 ps_main( PS_INPUT Input ) : COLOR0
{
    vec3 normal = normalize(tex2D(normalMap, Input.Texcoord).xyz * 2.0 - 1.0);
    float intensity = max(dot(normal, normalize(Input.LightDirection)), 0.0);
    return intensity;
}
```

- ① 텍스처에 블러 효과를 적용한다
- ② 노멀 맵을 사용하여 표면의 조명 강도를 계산한다
- ③ 물체의 윤곽선을 강조하는 쉘 셰이딩을 구현한다
- ④ 물 표면의 반사 효과를 시뮬레이션한다

10. 다음 HLSL 코드에서 성능 최적화 관점에서 가장 개선이 필요한 부분은?

```
float4 PS(float2 texCoord : TEXCOORD0) : SV_TARGET
{
    float4 color = float4(0, 0, 0, 0);

    for (int i = 0; i < 64; i++) {
        for (int j = 0; j < 64; j++) {
            float2 offset = float2(i / 640.0f, j / 480.0f);
            color += texture.Sample(texSampler, texCoord + offset) / 4096.0f;
        }
    }
    return color;
}
```

- ① 중점 루프를 사용한 텍스처 샘플링
- ② 상수 나눗셈 연산(/ 4096.0f)
- ③ float4 초기화 방식
- ④ 텍스처 좌표 오프셋 계산 방식

11. Vulkan은 DirectX 12와 함께 현대 게임 개발의 주요 API로 자리 잡았으며, 특히 고성능 게임 애플리케이션에서 많이 사용된다. Vulkan에 대한 아래 설명 중 틀린 것은?

- ① 저수준 API: 개발자에게 하드웨어에 대한 더 직접적인 제어 권한을 제공한다.
- ② 멀티스레딩 최적화: 현대 멀티코어 CPU를 효율적으로 활용할 수 있다.
- ③ 감소된 CPU 오버헤드: 물리 엔진 명령을 더 효율적으로 처리한다.
- ④ 크로스 플랫폼: 윈도우, 리눅스, 안드로이드, iOS 등 다양한 플랫폼을 지원한다.

12. 다음 코드는 2D 게임에서 타일맵을 구현하는 C 코드의 일부다. 코드를 분석하고, 보기 중 틀린 것을 고르시오.

```
#define MAP_WIDTH 64
#define MAP_HEIGHT 48
#define TILE_SIZE 32

typedef struct {
    int id;           // 타일 ID
    int solid;        // 충돌 여부 (0: 통과 가능, 1: 충돌)
    char* name;       // 타일 이름
} Tile;

typedef struct {
    Tile* tiles[MAP_WIDTH][MAP_HEIGHT]; // 타일맵 데이터
    int visible_tiles_x;                  // 화면에 표시되는 타일 수 (가로)
    int visible_tiles_y;                  // 화면에 표시되는 타일 수 (세로)
} TileMap;

// 타일맵 초기화 함수
void initTileMap(TileMap* map, int screen_width, int screen_height) {
    // 화면에 표시되는 타일 수 계산
    map->visible_tiles_x = screen_width / TILE_SIZE + 1;
    map->visible_tiles_y = screen_height / TILE_SIZE + 1;

    // 타일맵 초기화 (모든 타일을 NULL로 설정)
    for (int x = 0; x < MAP_WIDTH; x++) {
        for (int y = 0; y < MAP_HEIGHT; y++) {
            map->tiles[x][y] = NULL;
        }
    }
}

// 타일맵 렌더링 함수
void renderTileMap(TileMap* map, int camera_x, int camera_y) {
    // 카메라 위치에 따른 시작 타일 인덱스 계산
    int start_x = camera_x / TILE_SIZE;
    int start_y = camera_y / TILE_SIZE;

    // 화면에 표시할 타일 렌더링
    for (int x = start_x; x < start_x + map->visible_tiles_x; x++) {
        for (int y = start_y; y < start_y + map->visible_tiles_y; y++) {
            // 맵 범위 체크
            if (x >= 0 && x < MAP_WIDTH && y >= 0 && y < MAP_HEIGHT) {
                if (map->tiles[x][y] != NULL) {
                    // 화면 좌표 계산
                    int screen_x = (x * TILE_SIZE) - camera_x;
                    int screen_y = (y * TILE_SIZE) - camera_y;

                    // 타일 렌더링 함수 호출 (구현 생략)
                    renderTile(map->tiles[x][y]->id, screen_x, screen_y);
                }
            }
        }
    }
}

// 특정 위치의 타일이 충돌 가능한지 확인하는 함수
int isTileSolid(TileMap* map, int world_x, int world_y) {
    // 월드 좌표를 타일 좌표로 변환
    int tile_x = world_x / TILE_SIZE;
    int tile_y = world_y / TILE_SIZE;

    // 맵 범위 체크
    if (tile_x < 0 || tile_x >= MAP_WIDTH || tile_y < 0 || tile_y >= MAP_HEIGHT) {
        return 1; // 맵 외부는 충돌로 처리
    }

    // 해당 위치에 타일이 없으면 통과 가능
    if (map->tiles[tile_x][tile_y] == NULL) {
        return 0;
    }

    // 타일의 충돌 속성 반환
    return map->tiles[tile_x][tile_y]->solid;
}
```

- ① initTileMap 함수는 화면에 표시되는 타일 수를 계산할 때 경계 타일을 고려하여 1을 더한다.
- ② renderTileMap 함수는 카메라의 위치를 기준으로 화면에 보이는 타일만 렌더링하여 성능을 최적화한다.
- ③ isTileSolid 함수에서 맵 외부 영역은 항상 충돌로 처리된다.
- ④ 타일맵 데이터 구조는 포인터 배열을 사용하기 때문에 메모리를 동적으로 할당하고 해제할 필요가 없다.

13. 다음 중 3D 게임 물리 엔진의 성능 최적화 기법으로 적절하지 않은 것은?

- ① 시간 단계(Time Step)를 가변적으로 조정하여 빠른 움직임에서도 안정적인 시뮬레이션을 구현한다.
- ② 섬 분할(Island Partitioning)을 통해 서로 영향을 주지 않는 물체 그룹을 독립적으로 처리한다.
- ③ 휴면(Sleeping) 상태 감지를 통해 정적인 물체의 시뮬레이션을 일시 중단한다.
- ④ 모든 충돌 쌍에 대해 항상 정밀 충돌 검사(Narrow Phase)를 수행하여 정확도를 높인다.

14. VR 애플리케이션에서 '모션 시크니스(Motion Sickness)'를 줄이기 위한 프로그래밍 기법으로 가장 적절하지 않은 것은?

- ① 시야각(FOV)을 좁게 설정하여 더 많은 콘텐츠를 한 번에 보여준다.
- ② 프레임 레이트를 90fps 이상으로 유지하여 지연 시간을 최소화한다.
- ③ 사용자의 물리적 움직임과 가상 움직임 간의 불일치를 최소화한다.
- ④ 텔레포트 방식의 이동 시스템을 구현하여 연속적인 가속도를 피한다.

15. 다음 중 LOD 전환(LOD Transition)에서 발생하는 '팝핑(Popping)' 현상을 줄이기 위한 기법이 아닌 것은?

- ① 지오메트리 모핑(Geometry Morphing)을 사용하여 두 LOD 레벨 사이를 점진적으로 변환
- ② 카메라 거리에 관계없이 항상 최고 LOD를 유지하여 전환 자체를 방지
- ③ 알파 페이딩(Alpha Fading)을 통해 LOD 전환 시 투명도를 조절
- ④ 디더링(Dithering) 패턴을 사용하여 LOD 전환을 마스킹

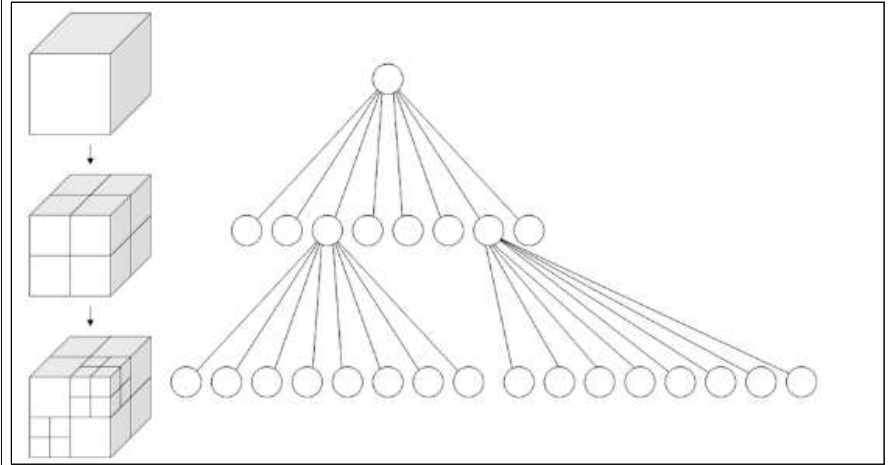
16. 3D 게임에서 스켈레탈 애니메이션 시스템에 관한 설명으로 옳지 않은 것은?

- ① 스켈레탈 애니메이션에서 '스키닝(Skinning)'은 뼈대의 움직임에 따라 메시의 버텍스들이 어떻게 변형될지 결정하는 과정이다.
- ② 애니메이션 블렌딩(Animation Blending)은 여러 애니메이션 간의 부드러운 전환을 위해 보간(interpolation)을 사용하며, 걷기에서 달리기로 자연스럽게 전환할 때 활용된다.
- ③ 역운동학(Inverse Kinematics, IK)은 최종 관절의 위치가 주어졌을 때 중간 관절들의 위치를 계산하는 기법으로, 캐릭터가 물체를 잡거나 발을 지면에 정확히 배치할 때 유용하다.
- ④ 스켈레탈 애니메이션에서 각 버텍스는 반드시 하나의 뼈에만 영향을 받아야 정확한 변형이 가능하다.

17. 1인칭 슈팅 게임(FPS)에서 부드러운 카메라 움직임을 위한 코드 구현에 관한 설명으로 옳지 않은 것은?

- ① 마우스 입력으로 카메라 회전을 구현할 때, 피치(상하 회전)에는 일반적으로 Y축 각도 제한(clamping)이 필요하며, 이는 캐릭터가 머리를 비현실적으로 회전하는 것을 방지한다.
- ② 총기 반동(recoil) 효과를 카메라에 적용할 때, 단순히 시야각(FOV)을 일시적으로 줄이는 것보다 카메라 위치에 perlin noise와 같은 절차적 노이즈를 적용하는 것이 더 현실적인 효과를 제공한다.
- ③ 1인칭 카메라와 3인칭 모델을 함께 사용하는 경우, 카메라 위치는 항상 캐릭터의 실제 머리 위치와 정확히 일치해야 하며, 어떠한 오프셋도 적용하면 안 된다.
- ④ 플레이어가 달리기를 할 때 적용하는 카메라 흔들림(head bobbing)은 sin 또는 cos 함수를 사용하여 구현하며, 걷기와 달리기 상태에 따라 진폭과 주기를 조절한다.

18. 보기 중 다음 그림에 대한 설명으로 알맞은 것은?



- ① 보기의 그림은 포털 컬링(Portal Culling)을 나타낸다. 이 기법은 주로 실내 환경에서 문이나 창문과 같은 '포털'을 통해 보이는 영역만 렌더링하여 성능을 최적화한다.
- ② 보기의 그림은 오클루전 컬링(Occlusion Culling)을 나타낸다. 이 기법은 다른 객체에 가려져 보이지 않는 객체를 식별하여 렌더링하지 않는 방식으로, 하드웨어 오클루전 쿼리나 소프트웨어 기반 가시성 결정 알고리즘을 사용한다.
- ③ 보기의 그림은 옥트리(Octree) 공간 분할을 나타낸다. 이 기법은 시야 절두체 밖에 완전히 위치한 객체나 노드를 렌더링에서 제외하며, 계층적 공간 분할을 통해 효율적으로 가시성을 결정한다.
- ④ 보기의 그림은 백페이스 컬링(Backface Culling)을 나타낸다. 이 기법은 카메라를 향하지 않는 폴리곤의 뒷면을 렌더링하지 않는 방식으로, 법선 벡터와 시선 벡터의 내적을 사용하여 결정한다.

19. 3D 게임에서 플레이어가 적 캐릭터를 향해 발사체를 발사하려고 합니다. 플레이어의 위치 벡터는 P(3, 2, 5)이고, 적 캐릭터의 위치 벡터는 E(7, 4, 8)입니다. 플레이어에서 적을 향하는 발사 방향 벡터를 계산하고 정규화하는 올바른 과정은?

단, D는 방향벡터, |D|는 벡터길이, D/|D|는 정규화벡터

- ① $D = P - E = (3, 2, 5) - (7, 4, 8) = (-4, -2, -3)$
 $|D| = \sqrt{(-4)^2 + (-2)^2 + (-3)^2} = \sqrt{16 + 4 + 9} = \sqrt{29}$
 $D/|D| = (-4, -2, -3) / \sqrt{29} \approx (-0.74, -0.37, -0.56)$
- ② $D = E - P = (7, 4, 8) - (3, 2, 5) = (4, 2, 3)$
 $|D| = \sqrt{4^2 + 2^2 + 3^2} = \sqrt{16 + 4 + 9} = \sqrt{29}$
 $D/|D| = (4, 2, 3) / \sqrt{29} \approx (0.74, 0.37, 0.56)$
- ③ $D = P \times E = (3, 2, 5) \times (7, 4, 8) = (2 \times 8 - 5 \times 4, 5 \times 7 - 3 \times 8, 3 \times 4 - 2 \times 7) = (16 - 20, 35 - 24, 12 - 14) = (-4, 11, -2)$
 $|D| = \sqrt{(-4)^2 + 11^2 + (-2)^2} = \sqrt{16 + 121 + 4} = \sqrt{141}$
 $D/|D| = (-4, 11, -2) / \sqrt{141} \approx (-0.34, 0.93, -0.17)$
- ④ $D = P \cdot E = (3 \times 7 + 2 \times 4 + 5 \times 8) = 21 + 8 + 40 = 69$
 $D/|D| = 69 / 69 = 1$

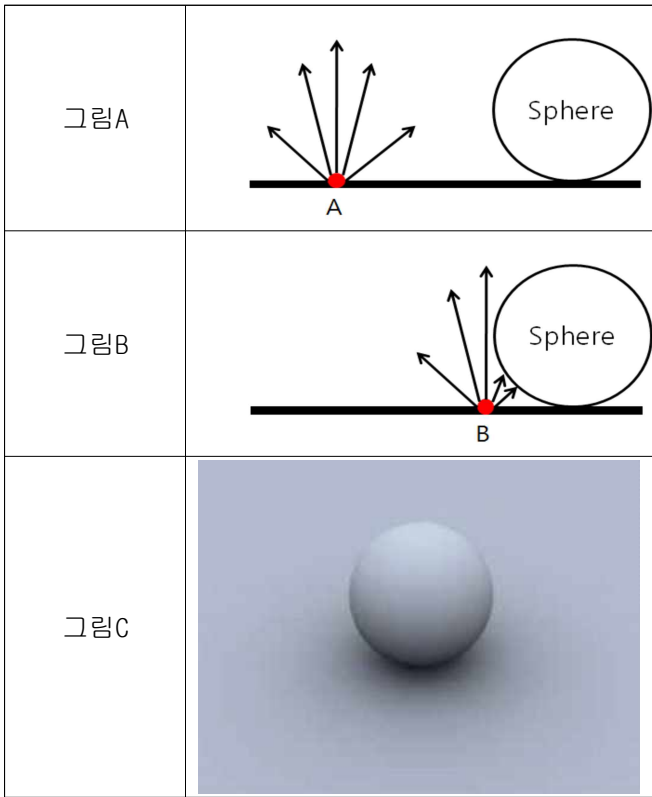
20. 절차적(Procedural) 지형 생성은 대규모 오픈 월드 게임에서 개발 시간을 단축하고 메모리 사용을 최적화하면서 다양한 환경을 효율적으로 제작할 수 있게 해주는 핵심 기술입니다. 3D 게임 개발에서 프로시저럴 지형 생성 알고리즘을 구현할 때, 다음 설명 중 프로그래밍 관점에서 옳지 않은 것은?

- ① 펄린 노이즈(Perlin Noise) 알고리즘을 구현할 때는 여러 옥타브(octave)의 노이즈를 중첩하는 방식으로 코딩하며, 각 옥타브마다 진폭(amplitude)은 감소시키고 주파수(frequency)는 증가시키는 방식으로 자연스러운 지형의 세부 정보를 생성한다. 보통 GPU 셰이더에서 구현하면 실시간 생성 성능이 크게 향상된다.
- ② 보로노이 다이어그램(Voronoi Diagram) 기반 지형 생성을 구현할 때는 먼저 무작위 시드 포인트를 생성하고, 각 격자점에서 가장 가까운 시드 포인트까지의 거리를 계산하는 방식으로 구현한다. 이를 활용하여 셀 기반의 크레이터, 바위 지형, 강 경계선 등을 효율적으로 생성할 수 있으며, 거리 함수를 변형하여 다양한 패턴을 만들 수 있다.
- ③ 하이트맵(Hightmap) 기반 지형 엔진에서 오버행(overhang)이나 동굴을 구현하려면, 각 (x,z) 좌표에 하나의 y값만 매핑하는 2D 배열 방식을 사용하고, 메시 생성 단계에서 특수한 텍스처 매핑과 노멀 맵을 활용하여 시각적으로 입체감을 구현하는 것이 메모리 효율성과 렌더링 성능 측면에서 가장 효과적이다.
- ④ L-시스템(L-System) 기반 지형 특성을 구현할 때는 문자열 치환 규칙과 재귀적 함수 호출을 포함한 알고리즘으로 코딩하며, 강 네트워크 생성 시에는 주 줄기에서 시작하여 분기 각도, 길이, 너비 등의 파라미터를 조절하여 자연스러운 수계 네트워크를 형성할 수 있다. 이때 지형의 경사를 고려한 경로 탐색 알고리즘을 함께 사용하면 더 현실적인 결과를 얻을 수 있다.

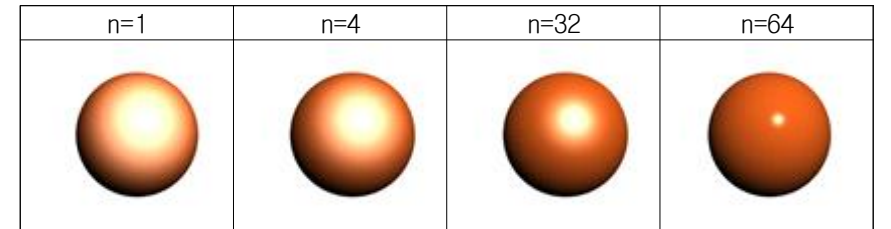
21. 다음 그림과 같이 원본 이미지를 흐릿하게 후처리하는 기술을 무엇이라 하는가?



- ① Embossing ② Edge Detection
③ Blur ④ Distortion
22. 다음 그림의 A 지점은 음영이 생길 확률이 0%지만, B는 40%(5개의 광선중 2개가 차폐됨)의 확률로 음영이 생길 수 있다. 이처럼 임의의 빛을 랜덤하게 발사하여 그 중 몇개가 가려지는지를 판단하여 음영을 만드는 전역조명 기술을 무엇이라 하는가?



- ① Ambient Occlusion ② Gouraud Shading
③ Phong Shading ④ Flat Shading
23. 다음 렌더링 파이프라인에서의 버퍼 중에서 화면에 출력할 픽셀들의 색상값을 저장하는 용도의 버퍼에 해당하는 것은?
- ① 인덱스 버퍼 ② 프레임 버퍼
③ 스텐실 버퍼 ④ 깊이 버퍼
24. 다음과 같이 n값이 증가할수록 표현범위가 좁아지는 조명을 무엇이라 하는가? 관련 수식은 다음과 같다. $I = (R \cdot V)^n$



- ① Ambient ② Diffuse
③ Specular ④ Emissive

25. 그림 A처럼 렌더링된 화면을 그림 B처럼 후처리하여 집중감을 향상시키고자 한다. 이러한 기법을 무엇이라 하는가? 그림 C는 처리하는 방법을 나타낸 것으로 먼저 픽셀 위치에서 블러 시작점 방향으로 샘플링하고, 시작점과 픽셀의 거리에 따라 Blur가중치를 계산한다.(멀수록 더 Blur시킨다)



- ① Euler Blur ② Radial Blur
③ Water Blur ④ Fluid Blur